

Replication

hxz390@student.bham.ac.uk

20 April 2026

1 Introduction

The following document is a step-by-step guide to help replicate the results of the neural network based configuration performance tuning algorithm as outlined in the corresponding report. The code being referenced is stored in the GitHub repo: https://github.com/harryz8/ISE_ToolBuildingProject. The baseline used is a program by the Ideas Lab stored in <https://github.com/ideas-labo/ise-lab-solution/tree/main/lab3>.

When running the solution, make sure that you are in the python virtual environment with source `venv/bin/activate` and then run the python files by doing `python3 {filename}` where filename is the file's name and extension.

2 Data

All of the test programs' configuration and performance data is sourced from the Ideas Lab at <https://github.com/ideas-labo/ise-lab-solution/tree/main/lab3> in the `datasets` folder. It is provided in the repository for this solution as well, also under a folder called `datasets`.

Table 1 shows a summary of the dataset characteristics for each program.

System	Language	Domain	Performance	Opts	Configs
7z	C++	Compressor	Runtime & Energy	8	68640
Brotli	C	Compressor	Runtime & Energy	2	180
LLVM	C	Compiler	Runtime & Energy	16	65536
PostgreSQL	C	Database	Runtime & Energy	8	864
x264	C	Video Encoder	Runtime & Energy	10	4608
Apache	C	Web Server	Runtime & Energy	8	640
Spear	C	SAT Solver	Runtime	14	16384
Storm	Clojure	Streaming Process	Latency	12	1557

Table 1: Subject system/program characteristics. *Opts* is the number of options, and *Configs* is the number of configurations.

3 Method

As explained in the report, this method of solving the configuration tuning problem has many hyperparameters. Before we can use this tuning algorithm, these values need to be set. The ones

that have been used are set in the file `configuration_tuning.py` but there was some process in finding as explained by the analysis below.

3.1 Hyper-parameter selection

Each hyper-parameter below has been changed independently of all others to produce two graphs, one against run time of our solution, and one against the absolute error of our produced optimal solution’s performance and the actual best configuration’s performance. Each graph contains at least 90 points representing independent runs of our solution for that program.

This is only possible in a test environment where finding the real optimum through brute force is possible. Buttons in the UI for this are therefore labelled as for test only. Hyper-parameter values must be chosen so that they work well across all the datasets while keeping in mind not to over-fit so that the model can generalise to different unseen datasets.

To find the optimal hyper-parameters, brute force search must be done across all possible values of all hyper-parameters, and hyper-parameters are likely to affect each other, creating a multi-dimensional landscape to be searched. This is too time consuming to optimise this way. The best method to solve this problem is to use a tuning software similar to the one outlined in this paper. Doing this seems contrary to the point of this paper, and hence these hyper-parameters are chosen in a less formal way: by picking sensible values that give a low error by trial and error and then creating some graphs as explained above plotting some tests of values around the sensible values for each hyper-parameter. This can then allow us to create trend graphs of the mean absolute error at each hyper-parameter value, giving us somewhat of a view as to how the hyper-parameters should be set. There is also a set of graphs of these values against time taken to get an optimal configuration rather than error for comparison, as we would likely have to find a trade off between these factors.

3.1.1 Initial Hyper-parameter Values

These graphs have been generated by varying one hyper-parameter in turn whilst keeping all of the others set to the values in table 2.

Hyper-parameter	Initial value
Evaluation size (<code>size_eval</code>)	10
Hidden layer size (<code>hidden_size</code>)	10
Epochs for evaluation batch (<code>epochs</code>)	300
Epochs for size 1 batch (<code>epochs_1batch</code>)	30
Learning rate (<code>learning_rate</code>)	0.01

Table 2: The initial choice of hyper-parameters

3.1.2 How to get the Graphs

1. Change the `hyperparameters` variable in `measure_hyperparameter_performance.py` to those in table 2. You can also set the budget here. To replicate the results, set it to 100.
2. Run `main.py` and click “Evaluate Hyperparameters” and then click “Clear all hyperparameter measurements”.

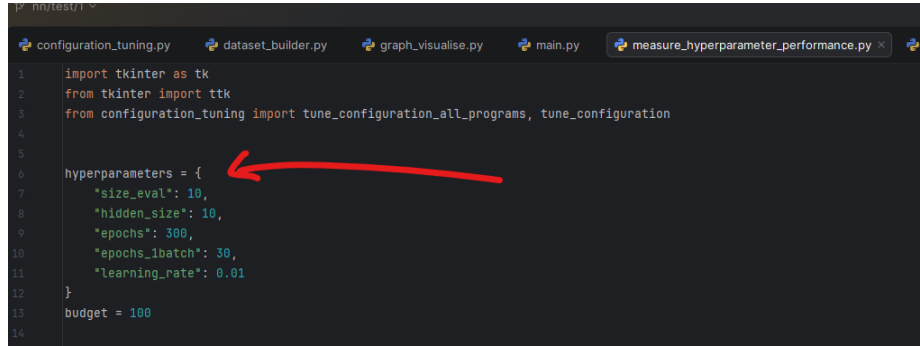


Figure 1: An image showing where the hyperparameters variable is in measure_hyperparameter_performance.py.

3. Now click “Measure hyperparameters” and then wait. This will generate the graphs by varying each hyper-parameter in turn whilst keeping the others fixed.
4. Set the `hyperparameters` variable in `configuration_tuning.py` to those in table 2. You can then view each graph by using the form to set the program and the budget and then choose the values for the x and y axes and press “Render Graph” (see figure 2). You could also save all of the graphs into the “./graphs” folder by clicking “Save all hyperparameter graphs”.

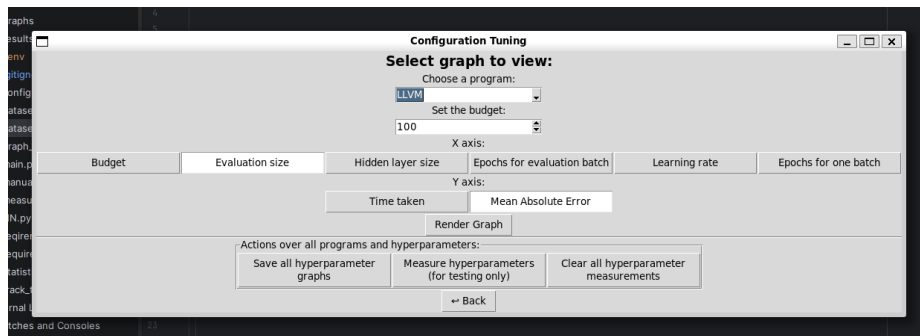


Figure 2: An image showing how to use the UI to render a hyper-parameter evaluation graph.

3.1.3 Analysis of the Graphs

Most of the graphs have the highest number of readings on these initial parameters, and as such these values have a higher chance of having outliers.

All of the 96 graphs are available in the program in the “Evaluate Hyper-parameters” section, but some examples are below.

1. **Evaluation Size** This refers to the amount of the budget that is used to create labelled training data for training the initial model.

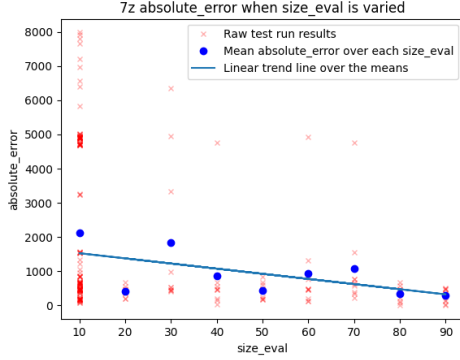


Figure 3: A graph of evaluation size against absolute error for the test dataset 7z.

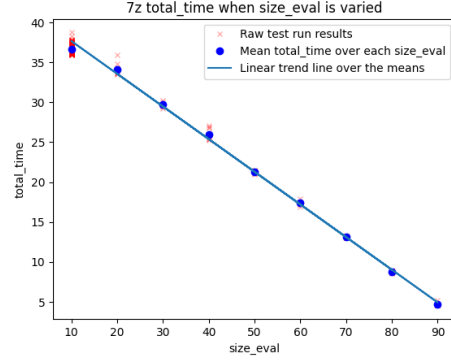


Figure 4: A graph of evaluation size against total time for the test dataset 7z.

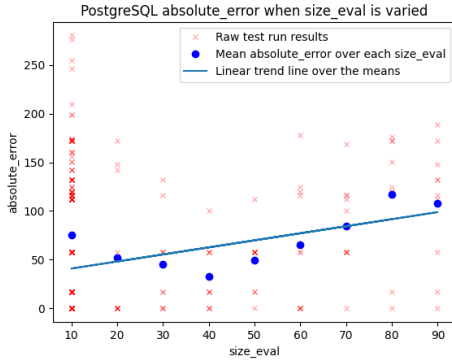


Figure 5: A graph of evaluation size against absolute error for the test dataset PostgreSQL.

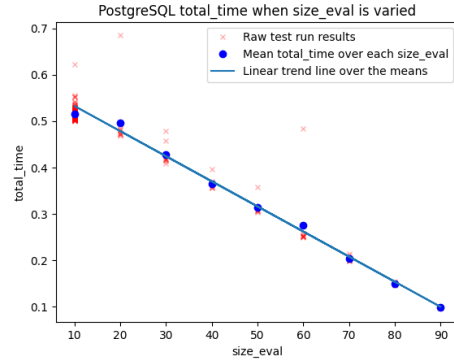


Figure 6: A graph of evaluation size against total time for the test dataset PostgreSQL.

All of the time graphs for each test program that we need to tune are reducing in time as evaluation size increases. This shows that for the initial hyper-parameter settings for the other hyper-parameters, a larger evaluation size would speed up the program. This is likely because, despite the initially chosen number of epochs for the evaluation batch being much greater than the initially chosen number of epochs for training on the single examples, it takes less time when more of the budget is used up in the parallelised section of the training process (when the evaluation size is large) than learning from each example in turn.

The evaluation size is an interesting hyper-parameter because increasing it affects the different programs in different ways. For example, in figure 3, you can see that the model is better on the 7z program when the evaluation size is increased, whereas figure 5 shows that the PostgreSQL program is negatively affected by an increase in the evaluation size. Many of the test programs' errors are pretty much not affected by the change in the evaluation size. The 7z program is the only one that is significantly positively affected by an increase, and this could be because its patterns are more difficult for the neural network to learn, meaning that the initial model

is much more inaccurate trained on a small evaluation training set, leading to bad predictions for the optimal configuration when using the rest of the budget, which can't rectify the initial disadvantage. Conversely, the programs which benefit from the smaller evaluation sizes would therefore have an initial model that is quite good at predicting and so the rest of the budget can be used for training on more optimal sections of the configuration space. These programs want this configuration tuning program to allocate the maximum budget to that later section.

Looking at the trend of the mean absolute errors in a less linear fashion reveals that for some of the programs being tuned there appears to be an optimum stationary point somewhere in the 30-50 range for evaluation size. The other benefit from increasing the evaluation size is the reduction in time taken, so the new evaluation size will be 30.

2. **Hidden Layer Size** This refers to the number of artificial neurons in the hidden layer of the neural network (the layer between the input and output layers).

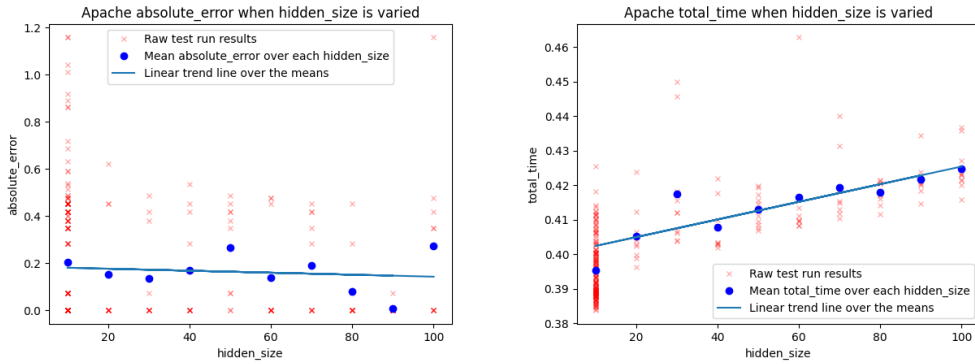


Figure 7: A graph of hidden layer size against absolute error for the test dataset Apache. Figure 8: A graph of hidden layer size against total time for the test dataset Apache.

All graphs of time taken to get the tuned configuration for all of the test programs that we are tuning increase in time as the hidden layer size increases. This is because, with more neurons, there are more weights to calculate and update.

The graphs of absolute error for hidden layer size all have either a very slow decreasing trend as in figure 7 or are pretty much not affected by the change in hidden layer size. This could be explained by the small budget of 100: with a small training set, a larger hidden layer may not be able to learn any more features than a smaller one. On larger budgets therefore, an increase of hidden layer size might help to reduce error.

The new hidden layer size will stay as 10 neurons to keep the time to tune a program low, but it may be useful to increase this if much larger budgets are being used.

3. **Epochs for Evaluation Batch** This refers to the number of times that the backwards pass is run and the weights are updated when training the initial model.

For most of the graphs the mean absolute error does not really change, but on some, such as for the program 7z as shown in figure 9, there is a clear increasing trend with more epochs. With more epochs, a traditional neural network becomes more specialised at giving the correct label for given training configurations, but beyond a certain value it will become less generalisable to

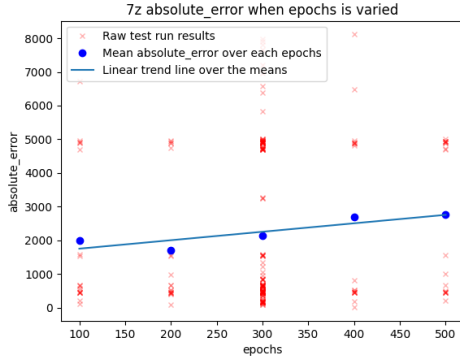
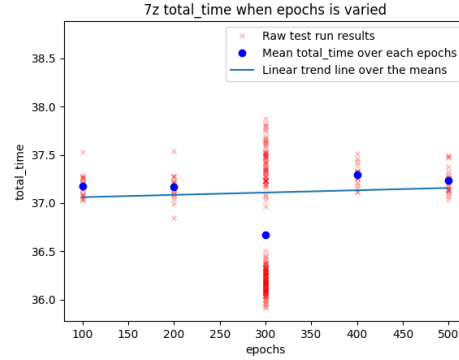


Figure 9: A graph of the number of epochs when training the initial "evaluation" training batch against absolute error for the test ing batch size against total time for the test dataset 7z.



unseen configurations. This is similar to the first part of our solution, where we train a neural network model on a small "evaluation" training set, using up part of the budget. If the model at this point is "less generalisable" (overfitted) then the next part in the process, predicting the minimum configuration and updating the model based on this value, may be quite a way off with a high error, and training these examples using the remainder of the budget may not be able to do enough to rectify this. This would be in contrast to a smaller number of epochs where the initial model has not had enough epochs to become overfitted. These particular programs for tuning, e.g. 7z, could be susceptible to this if they have a lot of local optimums which this version of gradient descent has a hard time jumping out of.

All of the graphs for each program to be tuned show an increase in time to tune with more epochs and this is understandable because the backwards step and the updating of weights have to be done more times.

The new value is 200 epochs to reduce the chance of over-fitting in the initial model and decrease the time taken to tune a program.

4. **Epochs for Size 1 Batch** This refers to the number of times that the backwards pass is run and the weights are updated when updating the model with a new piece of labelled data.

When the number of epochs for single new pieces of labelled data is increased, this means that the backwards pass is run and the weights are updated more times, leading two the tuning process taking longer overall which is reflected in the graphs such as figure 12 and 14.

For mean absolute error some programs that are being tuned get worse with an increase in epochs for single new pieces of labelled data, some get better and some don't change at all. One reason for this could be the different landscapes of each of the programs' configurations. On some, the landscape is such that extra training pushes it more towards the optimum, hence they respond well to an increase in this hyper-parameter. This landscape is likely smoother with shallower gradients. On others, the landscape is the opposite and it is much more difficult for the gradient descent process to find and descend into the global optimum. These may respond negatively to the increase in the number of epochs for single new pieces of labelled

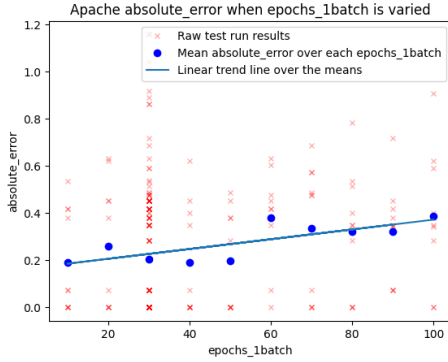


Figure 11: A graph of the number of epochs when updating the model with a single new piece of labelled data against absolute error for the test dataset Apache.

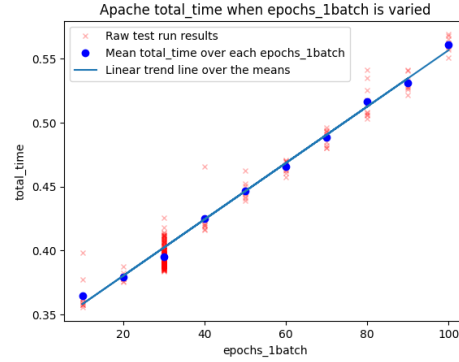


Figure 12: A graph of the number of epochs when updating the model with a single new piece of labelled data against total time for the test dataset Apache.

data in certain situations, such as when the learning rate is too high. This is an issue for such programs' configuration spaces because the steep gradients can cause the model to jump out of a minimum rather than head down, and this could be what is happening in some of these graphs, e.g. figure 11. Those which don't change could be explained by thinking that maybe they have reached a point where the model is fit pretty much perfectly and so the gradient doesn't really get updated.

Leave the value of this hyper-parameter for now.

5. **Learning Rate** This hyper-parameter controls how quickly the neural network model learns from the error of the previous pass. A smaller learning rate can be useful to improve the accuracy of linear descent, and avoid overshooting optimal weights. A lower learning rate often leads to slower convergence however.

For the learning rate, I have not chosen evenly spaced values for the learning rate, choosing to do more samples towards the smaller end, with the smallest being 0.0001. Each blue point is a mean across errors for a separate value of the learning rate.

Most graphs have lower error with larger learning rate. This could be because these programs' configuration space is relatively smooth, and so an increase in learning rate allows the model to learn more on the backwards step, similar to increasing the number of epochs, which can help the model to become better trained, hence the lower error. It also increases the chance of gradient descent overshooting local minima that it could otherwise get trapped in, potentially allowing the algorithm to find the global optimum more often. Graphs for the effect of learning rate on mean absolute error for test programs such as Apache (see figure 15) have a lower mean absolute error with a smaller learning rate. This suggests that these programs' configuration spaces are steeper around the global minimum because a larger learning rate is then able to overshoot and diverge away from the global minimum. There is more nuance however; many of these graphs show that some of the lowest options for learning rate that have been assessed have some of the largest mean absolute errors and they also often have the largest spread of different absolute error readings for different run results. This suggests that with a learning

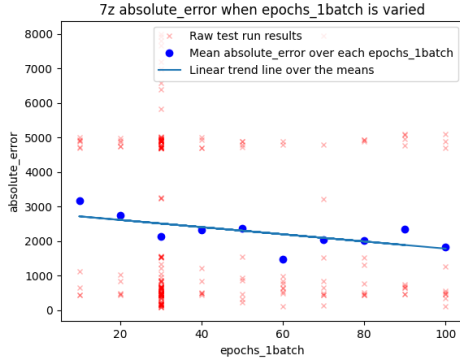


Figure 13: A graph of the number of epochs when updating the model with a single new piece of labelled data against absolute error for the test dataset 7z.

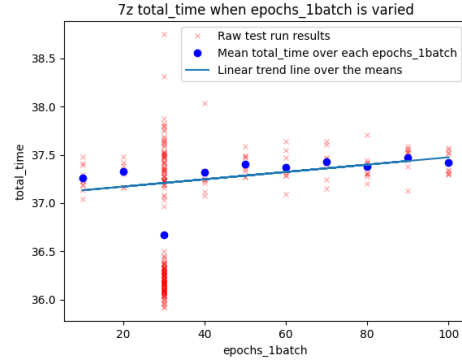


Figure 14: A graph of the number of epochs when updating the model with a single new piece of labelled data against total time for the test dataset 7z.

rate this low there is not enough epochs for this small of a learning rate to learn the underlying landscape well enough.

The time of the tuning process for a program should not change based on the learning rate because the number of epochs in both the initial model training section and the single reading training section are fixed as these graphs are made. Therefore variation seen in the graphs, such as a small increasing trend or a small decreasing trend must be down to noise.

The new learning rate is 0.003 which is slightly lower than the initial choice and this is to allow some of the more difficult datasets to train more accurately and so reduce their error. However increasing the learning rate means that the number of epochs should also be increased because the model now learns slower, so the new number of epochs for the initial "evaluation" batch will be 400 and the new number of epochs for size 1 batch is to be 90.

3.1.4 Final Hyper-parameter Values

Summarised in table 3.

Hyper-parameter	Final chosen value
Evaluation size (<code>size_eval</code>)	30
Hidden layer size (<code>hidden_size</code>)	10
Epochs for evaluation batch (<code>epochs</code>)	400
Epochs for size 1 batch (<code>epochs_1batch</code>)	90
Learning rate (<code>learning_rate</code>)	0.003

Table 3: The final choice of hyper-parameters

Note: Now close the UI window and set the `hyperparameters` variable in `configuration_tuning.py` to those in table 3.

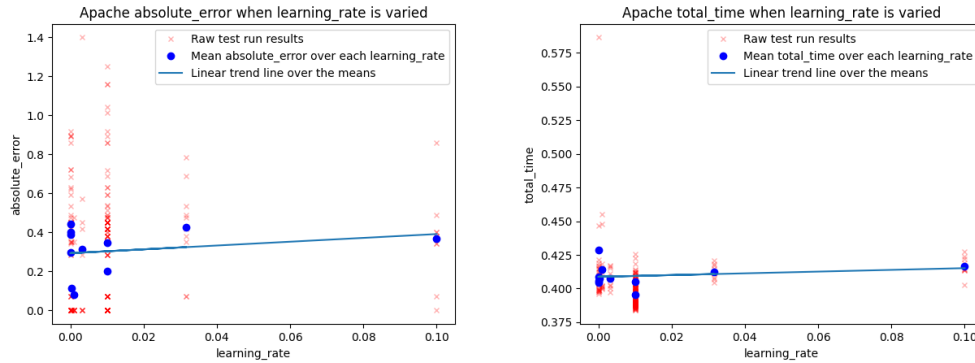


Figure 15: A graph of the learning rate against absolute error for the test dataset Apache. Figure 16: A graph of the learning rate against total time for the test dataset Apache.

3.2 Obtaining results for the test programs

1. Run `main.py` and click “Clear all results”, and then close the window.
2. Run `configuration_tuning.py` and choose a budget of 100 (or whatever you have been working with) and wait for it to tune each of the test programs 100 times each.

You can then view the results as graphs against iteration (see figure 19) for each program to be tuned by running `main.py` and setting the program and budget and then pressing “View Previous Results”, or you can save them all in the “./graphs” folder by pressing “Save all result graphs”.

You can view the results as histograms with outcomes grouped into 10 bins (see figure 20) because you can see them in the “./graphs” folder after running `graph_visualise.py`.

3.3 The Statistical Test

The statistical test being used is the Wilcoxon rank-sum test.

A hundred runs of the baseline for each of the programs that are being tuned can be found at <https://github.com/ideas-labo/ise-lab-solution/tree/main/lab3>.

1. Download the csv files of the baseline’s results.
2. Run `statistical_test.py` for each of the programs being tuned in turn and:
 - (a) Enter the path to the folder containing the csv results files made by this solution (it will be “./results”). You need to give the path from the current location which was told to you in the terminal window where you are running this.
 - (b) Then enter the filename of the results file made by this solution for the chosen program (but not the extension). It will be in the form “{program_name}_results” where you swap out {program_name} for the name of the program you chose (with the correct case; look in “./results” to double check).
 - (c) Next enter the path to the folder containing the csv results files made by the baseline that you downloaded. You need to give the path from the current location which was told to you in the terminal window where you are running this.

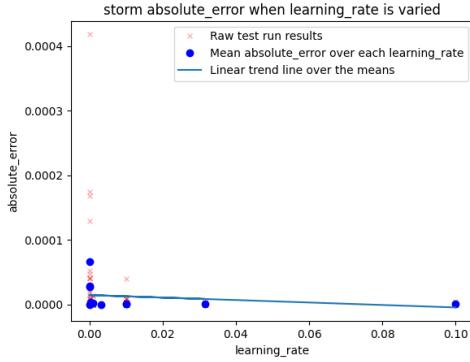


Figure 17: A graph of the learning rate against absolute error for the test dataset storm.

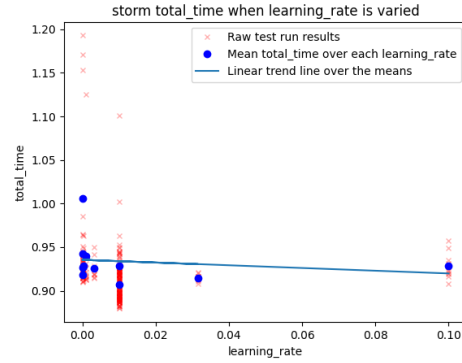


Figure 18: A graph of the learning rate against total time for the test dataset storm.

- (d) Finally, enter the filename of the baseline results file for the chosen program (but not the extension).
- (e) You will receive the p-value and the medians of both sets of results for this program.

You should have now obtained p-values and medians similar to those in table 4. They will likely differ because of the random weight initialisation of the neural network and the random configuration selection of the first part of the tuning algorithm where it trains an initial model.

Program	p value	Median of results from this solution	Median of results from the baseline solution
7z	3.23×10^{-31}	4700	35700
Apache	3.31×10^{-34}	31.0	337
brothli	1.13×10^{-33}	1.46	3.32
LLVM	2.52×10^{-34}	52300	76700
PostgreSQL	2.66×10^{-33}	46000	128000
spear	6.08×10^{-33}	0.000993	0.992
storm	3.11×10^{-34}	0.0	2.0
x264	7.76×10^{-34}	21.7	49.8

Table 4: Statistical test p value results with all numbers rounded to 3 significant figures

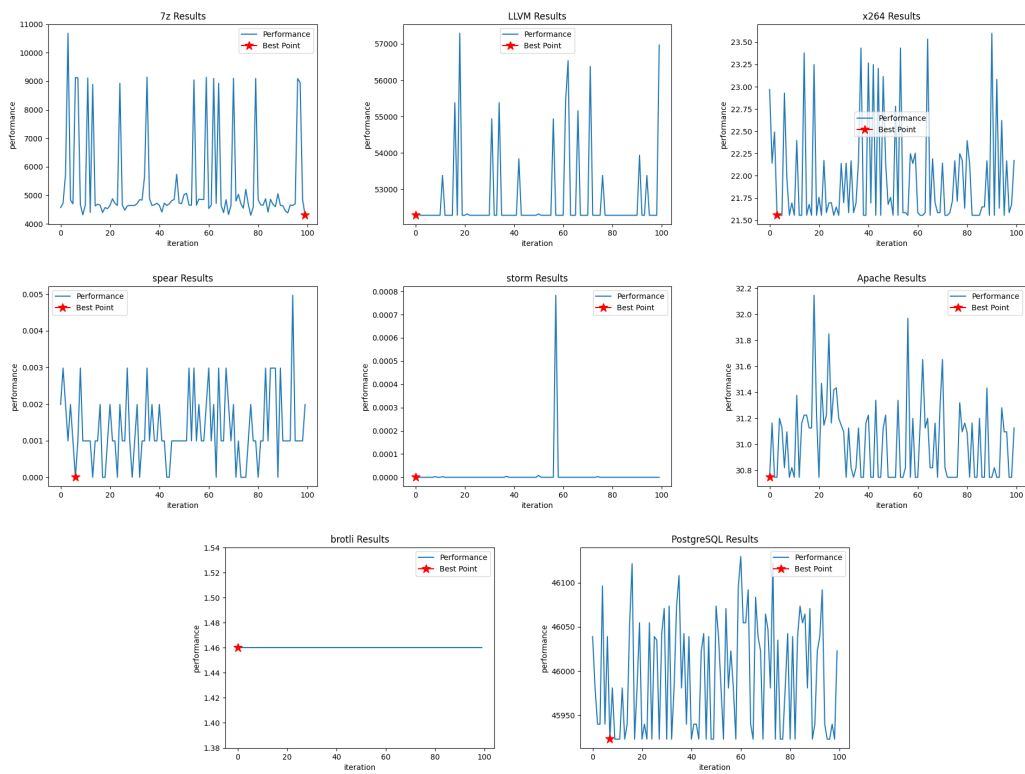


Figure 19: Graphs of tuned performance over 100 runs of the solution outlined in this assignment on each test program.

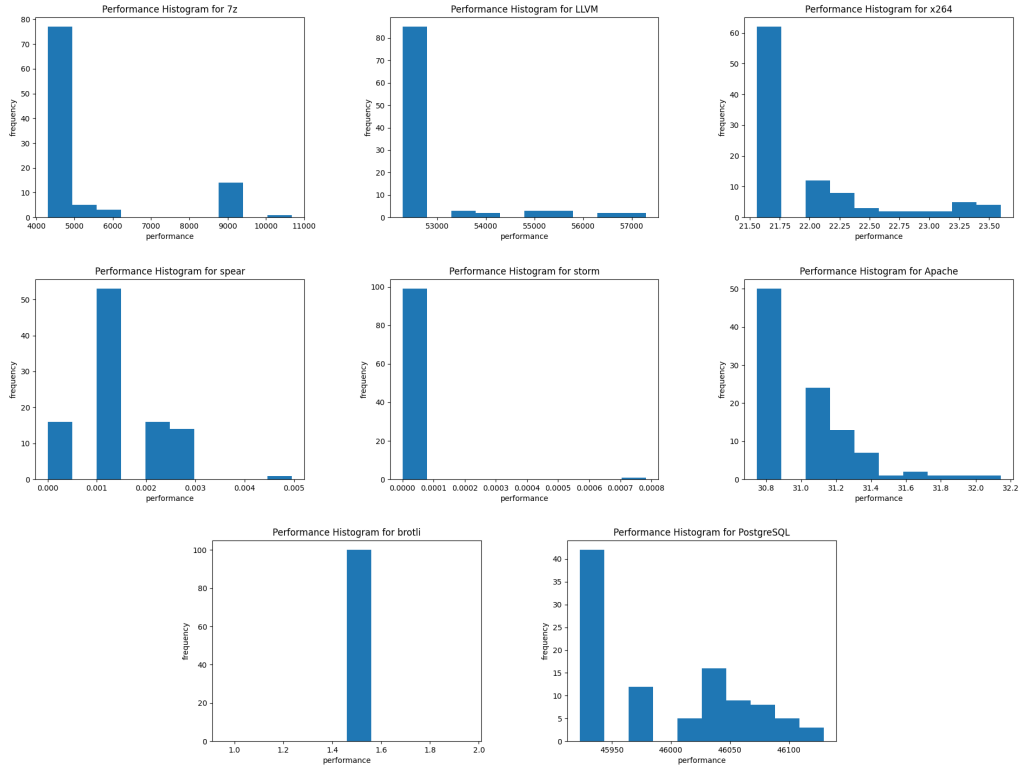


Figure 20: Graphs of frequency against tuned performance for 100 runs of the solution outlined in this paper on each test program.